

UNIVERSITY OF PERADENIYA

Department of Computer Engineering

Lessons Learnt from the Second Year Project

An expository article about the experience gathered from CO2060: Software Systems Design Project and recommendations for future enhancements

E/23/299 – Ranasinghe R. D. K. D.

September 2026

My Experience Working on the Project

The second-year project was one of the first opportunities where I had to work on a software system meant to solve a real problem instead of only completing an academic exercise. Our project, the Orthodontics Workflow Automation System (OWAS), was developed to support work in an orthodontic clinic by replacing scattered paper records and manual coordination with a centralized digital hub. The system handles patient records, appointments, clinical notes, and user access control, effectively reducing the average time to locate patient files from over 5 minutes to an instant search.

At the beginning, the project looked like a standard software development task. However, my specific responsibility—implementing the Live Clinic Queue section—taught me that building a system for real users requires much more than writing code. The clinic queue had to accurately mirror the complex, physical flow of patients moving between receptionists, nurses, students, and clinicians.

One important lesson I learnt was the absolute necessity of strict Role-Based Access Control (RBAC). In a healthcare-related system, not every user should see or change all information. For the clinic queue, I had to implement logic ensuring that administrators and nurses possessed global read-only access, clinicians had the authority to update queue statuses, and students could only access their specifically assigned local queue entries. Receptionists were given the unique permission to add patients to the live queue. Through this work, I understood why it is critical to enforce access control on both ends; hiding a button in the React frontend is helpful for the user interface, but the Express API routes and backend controllers must still actively reject unauthorized requests to maintain security.

Another lesson was the importance of team communication and version control workflows. Because the queue module interacted heavily with Patient Directory and User Management modules handled by other team members, maintaining system stability required constant coordination. I frequently had to resolve Git conflicts after syncing with the latest main branch, rigorously test the production frontend build, and submit my completed work

through formal GitHub pull requests. I learnt that sharing progress and understanding how different parts of the system interact saves significant time during deployment.

Testing was also a major learning experience. Because clinical staff and students frequently use tablets and mobile devices at dental units, touch scrolling integrity and interface stability were paramount. We relied on automated multi-device regression testing using Playwright to ensure the queue interface rendered correctly across different screen sizes (like iPads and Android tablets) and browsers. This made me realize that testing is not a final step done after development, but an ongoing requirement to ensure clinical usability.

Recommendations

For Staff and Instructors

Staff members should continue providing feedback based on real clinic workflows. Feedback from demonstrations is valuable because it helps us understand nuances—like how a patient transitions from "Waiting" to "Under Treatment"—that are difficult to identify from standard requirements alone. Before using the system fully, staff should agree on clear procedures for how patients are placed into the live queue and who is explicitly responsible for updating their status to prevent bottlenecks in the digital system.

It is also recommended to identify one or two responsible administrators for user management and basic system maintenance. They should regularly monitor the live queue for inactive sessions or stuck patients, review audit logs, and ensure that role assignments remain accurate. Short role-specific training sessions would be highly beneficial, as reception staff, nurses, and clinicians interact with the clinic queue in completely different ways.

For Future Batches

Future batches should begin by thoroughly studying the existing documentation, RBAC permission matrices, and Express API controllers before attempting to add new features to the queue. They should avoid changing the system without understanding the current workflow, as a minor change in the queue's backend authorization logic could accidentally expose restricted patient data to unauthorized student roles.

The next team could improve the project by adding more automated real-time capabilities to the queue (such as WebSockets for instant state updates without manual refreshing), improving mobile usability, and collecting user feedback after longer periods of real hospital use. They may also consider notification improvements so clinicians are immediately alerted when a patient is assigned to their local queue.

Most importantly, future batches should keep the documentation updated whenever they change a feature. The system will be much easier to maintain if the codebase, API endpoints, and permission descriptions remain consistent, ensuring the Orthodontics Workflow Automation System continues to be reliable for the clinic.